

Outline

What is Machine Learning?

Supervised Tasks: Regression vs. Classification

Data

Models and Parameters

Linear Regression with L_2 Loss

ML is changing our world

- ▶ Search engines learn your preferences
- ▶ Recommender systems learn your taste in books, music, movies
- ▶ Algorithms do automatic stock trading
- ▶ Translation between many languages
- ▶ DeepMind beats humans at Go
- ▶ Personalized medicine
- ▶ LLMs revolutionize many fields (especially coding)
- ▶ Data-driven discoveries in physics, biology, genetics, astronomy, neurology
- ▶ ...

AI, ML, and DL

- ▶ **Artificial Intelligence** — broad term; machines doing tasks that “require intelligence.” Includes ML, NLP, vision, robotics, planning, search.
- ▶ **Machine Learning** — mathematically well-defined; build predictive models *from data* instead of programming them.
- ▶ **Deep Learning** — subfield of ML focused on neural networks. Excellent on images, text, signals.

“A computer program is said to learn from experience E with respect to some task T and performance measure P , if its performance on T , as measured by P , improves with experience E .”

— Tom Mitchell, Carnegie Mellon, 1998

ML vs. Statistics

- ▶ Historically different fields, but methods and concepts heavily overlap
- ▶ **ML**: emphasis on accurate prediction with complex models
- ▶ **Stats**: emphasis on interpreting relationships and inference
- ▶ In practice: same problems, same tools, different vocabulary
- ▶ One useful framing: $ML \approx \textit{nonparametric statistics} + \textit{efficient numerical optimization}$

Three main directions of ML

- ▶ **Supervised learning** — data has labels y . Learn $f : \mathbf{x} \mapsto y$. Examples: predict house price, classify image. [Focus of this course.](#)
- ▶ **Unsupervised learning** — no labels. Find structure in $\mathbf{x}$. Examples: clustering, dimensionality reduction, anomaly detection.
- ▶ **Reinforcement learning** — agent interacts with environment. Receives reward signal. Learns to choose actions that maximize future reward.

Regression vs. Classification

Supervised learning splits by the type of the target y :

- ▶ **Regression:** $\mathcal{Y} \subseteq \mathbb{R}$ — predict a number.
 - ▶ House prices, temperature, length of hospital stay
- ▶ **Classification:** $\mathcal{Y} = \{C_1, \dots, C_g\}$ — predict a category.
 - ▶ Spam vs. not spam, image of cat/dog/bird, risk category

Both share the same machinery: features $\mathbf{x}$, labels y , a model f , a loss.

Two reasons to learn a relationship

- ▶ **Learning to predict:** we want accurate predictions on new data. Internal structure of f may not matter.
- ▶ **Learning to explain:** we use f as a window into the data-generating process. Predictive accuracy still matters, but interpretability is the goal.

ML historically emphasized prediction; classical statistics emphasized explanation. Most modern work cares about both, to varying degrees.

Tabular data in supervised learning

n observations of p features plus a target:

- ▶ **Target** (y) — the variable we want to predict
- ▶ **Features** ($\mathbf{x}$) — variables that describe each object

Iris example (Fisher, 1936): 150 flowers, 4 features (sepal length/width, petal length/width), 3 species. Tiny, clean, very easy to classify — *not* representative of real data.

Data types

Features and target may be of different types:

- ▶ **Numerical:** $\in \mathbb{R}$
 - ▶ **Integer:** $\in \mathbb{Z}$
 - ▶ **Categorical:** $\in \{C_1, \dots, C_g\}$
 - ▶ **Binary:** $\in \{0, 1\}$
-
- ▶ Target type determines the task: regression vs. classification
 - ▶ Most learning algorithms need **numerical** features. Categoricals are usually **encoded** first (trees are a notable exception).

Encoding categorical features

A categorical x with k values becomes a length- $\tilde{k}$ indicator vector:

- ▶ **One-hot encoding:** $\tilde{k} = k$ dummies; exactly one is 1.
Example: $x \in \{a, b, c\}$, $x = c \mapsto (0, 0, 1)$
- ▶ **Dummy encoding:** $\tilde{k} = k - 1$ dummies; at most one is 1. Removes redundancy (needed when the design matrix must be non-singular, e.g., linear regression).

Ordinal features (small < medium < large) get an order-preserving encoding (e.g., 0, 1, 2).

Notation for data

A dataset of size n :

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(1)}, y^{(1)} \right), \dots, \left(\mathbf{x}^{(n)}, y^{(n)} \right) \right\} \in (\mathcal{X} \times \mathcal{Y})^n$$

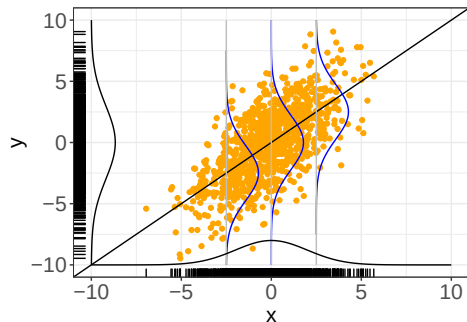
where

- ▶ $\mathcal{X} \subseteq \mathbb{R}^p$ — input (feature) space
- ▶ $\mathcal{Y}$ — output (target) space
- ▶ $(\mathbf{x}^{(i)}, y^{(i)})$ — i -th observation
- ▶ $\mathbf{x}_j^{(i)}$ — j -th feature of i -th observation

$\Rightarrow n$ observations, each described by p features.

Data-generating process

- ▶ Data $\mathcal{D}$ generated by some distribution $\mathbb{P}_{xy}$ over $\mathcal{X} \times \mathcal{Y}$
- ▶ $\mathbb{P}_{xy}$ is **unknown** — learning its structure is what ML is about
- ▶ Assume **i.i.d.**: samples drawn independently from the same distribution
- ▶ Strong assumption — fails for time series, spatial data. We focus on the i.i.d. case.



What is a model?

A **model** (or **hypothesis**) is a function

$$f : \mathcal{X} \rightarrow \mathbb{R}^g$$

mapping feature vectors to predicted target values.

- ▶ Regression: $g = 1$
- ▶ Classification: $g = \text{number of classes}$; outputs are scores or probabilities
- ▶ Models range from very simple (linear, tree stump) to very complex (deep networks)
- ▶ ML requires **constraining** f to a chosen family of functions

Hypothesis spaces

- ▶ Without restricting the function family, finding a “good” f is hopeless
- ▶ We declare the model class *a priori*; this is a **structural prior**
- ▶ The **hypothesis space** $\mathcal{H}$ is the set of all admissible functions:

$$\mathcal{H} = \{f : f \text{ belongs to a chosen functional family}\}$$

Parameterization

- ▶ Models in $\mathcal{H}$ usually share a structure parameterized by θ
- ▶ Parameter vector $\theta = (\theta_1, \dots, \theta_d) \in \Theta$
- ▶ Setting θ fixes a specific function from the family

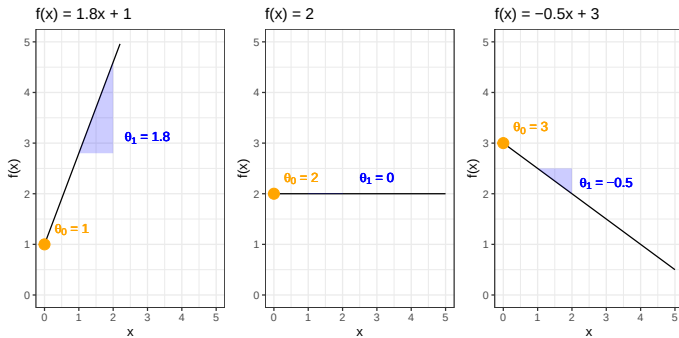
$$\mathcal{H} = \{f_\theta : f_\theta \text{ in chosen family, parameterized by } \theta \in \Theta\}$$

Finding the optimal model $\Leftrightarrow$ **finding the optimal parameters** — search on a d -dimensional surface.

Example: univariate linear functions

$$\mathcal{H} = \{f : f(x) = \theta_0 + \theta_1 x, \theta \in \mathbb{R}^2\}$$

Two parameters: intercept θ_0 and slope θ_1 . Each choice of (θ_0, θ_1) picks one line from the family.



Example: bivariate quadratic functions

$$\mathcal{H} = \{f : f(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2, \boldsymbol{\theta} \in \mathbb{R}^6\}$$

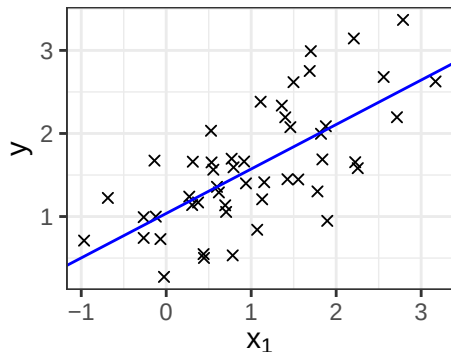
6 parameters. Adding terms (the squared and cross terms) lets the surface curve. More parameters $\Rightarrow$ richer hypothesis space, but also more risk of overfitting (next lectures).

Linear regression

Predict $y \in \mathbb{R}$ as a **linear** combination of features:

$$\hat{y} = f(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p$$

- ▶ $\mathcal{H} = \{f : f(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}, \boldsymbol{\theta} \in \mathbb{R}^{p+1}\}$
- ▶ Find loss-optimal $\boldsymbol{\theta}$ describing $y \mid \mathbf{x}$
- ▶ Special case of a linear model in arbitrary *basis functions* (polynomials — see L02)



Design matrix

- ▶ Mismatch: $\boldsymbol{\theta} \in \mathbb{R}^{p+1}$ vs. $\mathbf{x} \in \mathbb{R}^p$ because of the intercept
- ▶ **Trick:** pad each $\mathbf{x}$ with a leading 1:

$$\mathbf{x} \mapsto (1, x_1, \dots, x_p)^\top \quad \Rightarrow \quad \boldsymbol{\theta}^\top \mathbf{x} = \theta_0 \cdot 1 + \theta_1 x_1 + \dots + \theta_p x_p$$

- ▶ Stack all observations into the **design matrix** $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} = \begin{pmatrix} 1 & x_1^{(1)} & \dots & x_p^{(1)} \\ 1 & x_1^{(2)} & \dots & x_p^{(2)} \\ \vdots & \vdots & & \vdots \\ 1 & x_1^{(n)} & \dots & x_p^{(n)} \end{pmatrix} \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{pmatrix}$$

Effect interpretation

- ▶ Big advantage of the linear model: **interpretable** feature effects.
- ▶ “Increasing x_j by 1 unit, holding all others fixed, increases $\hat{y}$ by θ_j units.”
→ *ceteris paribus* reading.
- ▶ Watch out: this interpretation breaks if features are strongly correlated, or if you transform features (e.g., log, squares).

Model fit: the L_2 loss

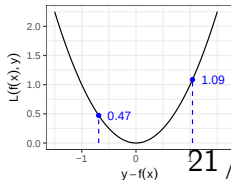
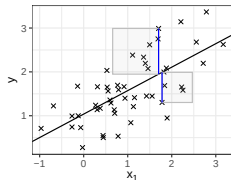
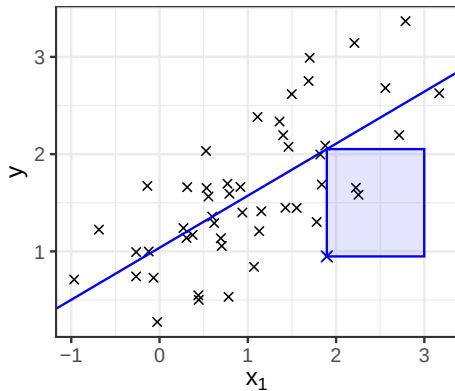
Pick the “best” θ by defining a **loss** and minimizing.

Most popular loss for regression — the L_2 / **quadratic** / **squared error**:

$$L(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2$$

Why quadratic in residuals $r = y - f(\mathbf{x})$?

- ▶ Convex and differentiable
- ▶ Closed-form solution for LMs
- ▶ Connects to classical stats (Gaussian noise)



From loss to empirical risk

The **empirical risk** sums the loss over the training set:

$$\mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \sum_{i=1}^n L\left(y^{(i)}, f\left(\mathbf{x}^{(i)}\right)\right) = \sum_{i=1}^n \left(y^{(i)} - \boldsymbol{\theta}^\top \mathbf{x}^{(i)}\right)^2$$

Also called **sum of squared errors (SSE)**.

Optimization:

$$\hat{\boldsymbol{\theta}} \in \arg \min_{\boldsymbol{\theta}} \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

Analytical optimization: the normal equations

Setting the gradient to zero,

$$\frac{\partial \mathcal{R}_{\text{emp}}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = 0,$$

gives the **normal equations**. Solving them yields the **ordinary least-squares (OLS)** estimator:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- ▶ Exists when $\mathbf{X}^\top \mathbf{X}$ is invertible (i.e., features are linearly independent)
- ▶ For most ML losses no such closed form exists $\Rightarrow$ we use gradient descent (next lecture)

Connection to classical statistics

Linear regression with L_2 loss = the classical statistics linear model under:

- ▶ Observations $\mathbf{x}^{(i)}$ are i.i.d.
- ▶ **Homoskedastic** Gaussian errors:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$$

so $\mathbf{y} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\theta}, \sigma^2 \mathbf{I})$

- ▶ Uncorrelated features (else: multicollinearity)

Under these, classical inference applies: p -values, t -tests, confidence intervals, R^2 .

Template: this is what most of ML looks like

We just saw the basic ML recipe:

1. Choose a hypothesis space $\mathcal{H} = \{f_{\theta}\}$
2. Choose a loss $L(y, f(\mathbf{x}))$
3. Define the empirical risk $\mathcal{R}_{\text{emp}}(\theta) = \sum_i L(y^{(i)}, f(\mathbf{x}^{(i)} | \theta))$
4. Find $\hat{\theta} = \arg \min_{\theta} \mathcal{R}_{\text{emp}}(\theta)$ — by closed form, gradient descent, or other

L02 — alternative losses (L1, robust). L03 — what happens when we want fewer parameters (regularization). L04/L05 — evaluating and tuning.